

Vertex Plugin Specification

Version: 1.0

Status: Draft

1. Objetivo

Este documento define a especificação oficial dos plugins do Vertex.

Um plugin é a unidade fundamental de funcionalidade da plataforma. Toda funcionalidade visível ao usuário é implementada como plugin.

Este documento estabelece o modelo, a estrutura, o ciclo de vida, o manifesto e as boas práticas para o desenvolvimento de plugins.

2. O que é um Plugin

No Vertex:

«Todo aplicativo é um plugin.»

Um plugin é um pacote autocontido que adiciona funcionalidades à plataforma utilizando exclusivamente as APIs públicas fornecidas pelo Core.

Plugins podem:

- fornecer interfaces gráficas;
- registrar capabilities;
- publicar eventos;
- consumir estados;
- executar tarefas em segundo plano;
- utilizar recursos do Android por meio do Core.

Plugins não possuem acesso direto às implementações internas do Core nem de outros plugins.

3. Objetivos

O modelo de plugins foi projetado para oferecer:

- modularidade;
- isolamento;

- baixo acoplamento;
- facilidade de atualização;
- segurança;
- reutilização;
- escalabilidade.

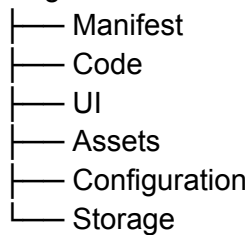
4. Estrutura Geral

Todo plugin é composto pelos seguintes elementos:

- Manifest;
- Código;
- Recursos (Assets);
- Interface;
- Configurações;
- Armazenamento privado;
- Metadados opcionais.

Estrutura simplificada:

Plugin



5. Manifest

Todo plugin deve possuir um Manifest, que descreve suas características e necessidades.

O Manifest inclui, entre outros:

- identificador único;
- nome;
- versão;
- autor;
- descrição;
- versão mínima do Core;
- permissões;
- capabilities fornecidas;
- capabilities utilizadas;
- eventos publicados;

- eventos assinados;
- estados publicados;
- estados consumidos;
- componentes internos;
- prioridade de inicialização;
- plugins dependentes;
- assinatura digital.

O Manifest é utilizado pelo Core durante:

- instalação;
- validação;
- carregamento;
- atualização;
- execução.

6. Estrutura Interna

Internamente, um plugin pode ser organizado em módulos.

Exemplo:

Plugin

UI

Services

Capabilities

Events

Storage

Assets

Configuration

Essa organização é recomendada, mas não obrigatória.

7. Plugins Componentizados

Quando um plugin ultrapassar as métricas definidas pelo Core (complexidade, tamanho ou quantidade de responsabilidades), ele poderá ser dividido em componentes internos.

Essa divisão melhora:

- manutenção;
- carregamento;
- reutilização;
- desempenho.

O processo é transparente para o restante da plataforma.

8. Plugin Context

Todo plugin recebe um PluginContext durante sua execução.

O PluginContext fornece acesso às APIs públicas da plataforma.

Exemplos de serviços disponíveis:

- Scheduler;
- Storage;
- Logger;
- Resource Manager;
- State;
- Events;
- Capabilities;
- Configuration;
- Assets;
- Android Bridge.

Nenhum serviço interno é acessado diretamente.

9. Plugin API

Toda comunicação entre o plugin e o Core ocorre através da Plugin API.

Ela representa o contrato oficial da plataforma.

Mudanças incompatíveis devem respeitar a política de versionamento.

10. Ciclo de Vida

Todo plugin segue o mesmo ciclo de vida.

Estados principais:

1. Installed
2. Validated
3. Loaded
4. Initialized
5. Running
6. Suspended
7. Hibernated
8. Resumed
9. Stopped
10. Unloaded
11. Removed

O Core controla todas as transições entre estados.

11. Comunicação

Plugins nunca acessam diretamente outros plugins.

A comunicação oficial ocorre através de:

- Capabilities;
- Events;
- State;
- Shared Objects.

Esse modelo reduz dependências e facilita evolução.

12. Segurança

Todo plugin executa dentro de uma sandbox controlada.

O acesso a recursos depende das permissões declaradas no Manifest.

Permissões podem ser concedidas, negadas ou revogadas pelo Core conforme as políticas da plataforma.

13. Recursos

O consumo de recursos é controlado pelo Resource Manager.

Cada plugin recebe um Budget para:

- CPU;
- memória;
- bateria;
- armazenamento;
- tarefas em segundo plano.

O Core pode limitar ou suspender plugins que excedam esses limites.

14. Atualização

Plugins podem ser atualizados independentemente do Core.

Durante a atualização:

- o Manifest é validado;
- compatibilidade é verificada;
- estado pode ser preservado quando possível;
- rollback pode ser realizado em caso de falha.

15. Versionamento

Cada plugin possui:

- versão do plugin;
- versão do Manifest;
- versão da Plugin API;
- versão mínima do Core;
- versão máxima suportada (opcional).

O objetivo é preservar compatibilidade entre diferentes versões da plataforma.

16. Falhas e Recuperação

Caso ocorra uma falha:

- o erro é isolado;
- logs são registrados;
- recursos são liberados;
- o plugin pode ser reiniciado;

- o restante da plataforma continua funcionando.

17. Plugin Inspector

O Plugin Inspector fornece informações sobre cada plugin.

Exemplos:

- estado atual;
- consumo de CPU;
- memória utilizada;
- bateria consumida;
- permissões;
- capabilities;
- eventos;
- estados;
- versão;
- dependências.

Essas informações auxiliam diagnóstico e depuração.

18. Boas Práticas

Plugins devem:

- possuir responsabilidade única;
- utilizar apenas APIs públicas;
- minimizar consumo de recursos;
- evitar bloqueios em threads principais;
- publicar apenas eventos necessários;
- documentar capabilities e estados;
- tratar exceções adequadamente;
- liberar recursos quando não utilizados;
- respeitar os Budgets definidos pelo Core.

Sempre que possível:

- reutilizar serviços existentes;
- evitar duplicação de código;
- manter baixo acoplamento;
- priorizar simplicidade.

19. O que um Plugin não deve fazer

Plugins não devem:

- acessar implementações internas do Core;
- acessar diretamente outros plugins;
- utilizar APIs privadas da plataforma;
- ignorar o ciclo de vida definido pelo Core;
- contornar o sistema de permissões;
- consumir recursos além dos limites estabelecidos.

20. Resumo

Os plugins representam a principal unidade de extensão do Vertex.

Ao centralizar todas as funcionalidades em plugins padronizados, a plataforma obtém uma arquitetura modular, segura, evolutiva e de fácil manutenção, permitindo que novas capacidades sejam adicionadas sem modificar o Core e preservando desempenho, estabilidade e compatibilidade ao longo do tempo.